

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №4

по курсу «Разработка кроссплатформенных приложений»

на тему «Работа с файлами»

Вариант 9

Выполнили студенты группы 23ВВИ2:

Юров Д.М.

Лисов Е.А.

Кочегин В.В.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Карамышева Н.С.

Пенза 2026

Цель работы: изучить работу с файлами и механизмы сериализации данных.

Задание: модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла («JFileChooser»).

Ход работы:

1. Модификация программы

В графическом интерфейсе были добавлены 4 новые кнопки – две для работы с файлами с данными в «нормальном» виде и две для работы с файлами с данными в двоичном виде с использованием механизма сериализации (рис. 1).

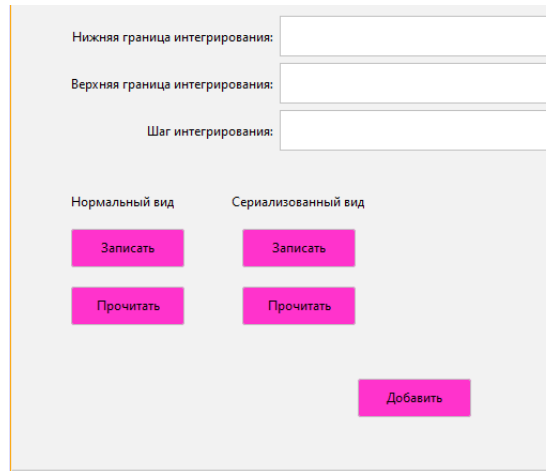


Рисунок 1 - модифицированный графический интерфейс

Был модифицирован класс «ResIntegral», после чего объекты этого класса могут быть сохранены/восстановлены стандартным образом (рис. 2).

```
public static class ResIntegral implements Serializable {
    public static final double MIN = 0.000001;
    public static final double MAX = 1000000.0;

    private double a;
    private double b;
    private double h;
    private Double result;

    public ResIntegral(double a, double b, double h) throws OutOfLimitException {
        if (a < MIN || b > MAX) {
            throw new OutOfLimitException("Неверные значения a и b - числа должно быть в диапазоне: 0,000001-1000000");
        }

        this.a = a;
        this.b = b;
        this.h = h;
    }

    public double getA() { return this.a; }
    public double getB() { return this.b; }
    public double getH() { return this.h; }
    public Double getRes() { return this.result; }

    public void setA(double num) { this.a = num; }
    public void setB(double num) { this.b = num; }
    public void setH(double num) { this.h = num; }
    public void setRes(double num) { this.result = num; }
}
```

Рисунок 2 - модифицированный класс "ResIntegral"

Была добавлена функция («writeFile») для записи файла в форматированном виде. Принимает в качестве аргументов объект класса File для получения информации о файле, в который будут записаны данные и массив «ArrayList» объектов класса «ResIntegral», который будет записан в файл. Запись осуществляется с помощью «FileWriter» внутри «BufferedWriter», чтобы запись происходила за один раз, то есть без множественных обращений к файлу – буфер накапливает информацию внутри себя и пишет всё за один раз во время заполнения буфера или вызова метода «flush» (принудительный сброс буфера в файл). Запись данных осуществляется с определенным форматом, для удобства чтения файла человеком, а для избежания конфликтов кодировок чтения и записи явно указаны кодировки и локали записи. После записи информации в файл буфер закрывается методом «close» (рис. 3).

```
491 private void writeFile(File file, ArrayList<Integration.ResIntegral> list)
492 {
493     BufferedWriter bw = null;
494     try
495     {
496         bw = new BufferedWriter(new FileWriter(file, StandardCharsets.UTF_8));
497         bw.write("#format: a;b;h;result");
498         bw.newLine();
499         for(Integration.ResIntegral e : list)
500         {
501             String a = String.format(Locale.US, "%.10f", e.getA());
502             String b = String.format(Locale.US, "%.10f", e.getB());
503             String h = String.format(Locale.US, "%.10f", e.getH());
504             String res;
505             if(e.getRes() != null)
506             {
507                 res = String.format(Locale.US, "%.10f", e.getRes());
508             }
509             else
510             {
511                 res = "";
512             }
513             bw.write(a + ";" + b + ";" + h + ";" + res);
514             bw.newLine();
515         }
516         bw.flush();
517         bw.close();
518     }
519     catch (IOException ex)
520     {
521         ex.printStackTrace();
522     }
523 }
```

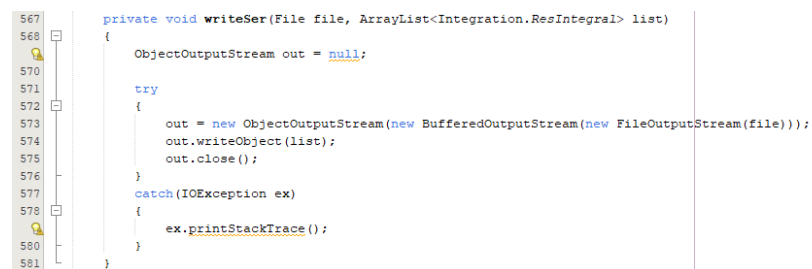
Рисунок 3 - функция "writeFile"

Была добавлена функция («readFile») для чтения файла в форматированном виде. Функция принимает на вход объект класса «File» для получения информации о файле, из которого будет происходить чтение информации и возвращает массив «ArrayList» объектов класса «ResIntegral». Чтение осуществляется с помощью «FileReader» внутри «BufferedReader» - связка работает так же, как и с записью. Первая строка файла считывается без запоминания – пропуск информации о записанных данных, остальные строки считываются в цикле «while», в котором каждый аргумент проходит проверку с помощью функции «parse» (рис. 4).

```
525 private ArrayList<Integration.ResIntegral> readFile(File file)
526 {
527     ArrayList<Integration.ResIntegral> list = new ArrayList<>();
528     BufferedReader br = null;
529     try
530     {
531         br = new BufferedReader(new FileReader(file, StandardCharsets.UTF_8));
532         String line;
533         line = br.readLine();
534         int i = 0;
535         while((line = br.readLine()) != null)
536         {
537             i++;
538             String[] p = line.split(";");
539
540             double a = parse(p[0], "Нижняя граница");
541             double b = parse(p[1], "Верхняя граница");
542             double h = parse(p[2], "Шаг");
543
544             Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);
545
546             if(p.length == 4)
547             {
548                 String resStr = p[3].trim();
549                 resStr = resStr.replace(',', '.');
550                 e.setRes(Double.parseDouble(resStr));
551             }
552             list.add(e);
553         }
554     }
555     catch(IOException ex)
556     {
557         ex.printStackTrace();
558     }
559     catch(OutOfLimitException e)
560     {
561         e.printStackTrace();
562     }
563     return list;
564 }
565 }
```

Рисунок 4 - функция "readFile"

Была добавлена функция («writeSer») для записи файла в двоичном виде с использованием механизма сериализации. Функция принимает в качестве аргументов объект класса «File» и массив «ArrayList» объектов класса «ResIntegral». Запись осуществляется с помощью «ObjectOutputStream» внутри «BufferedOutputStream» внутри «FileOutputStream». «ObjectOutputStream» - поток, отвечающий за преобразование объектов в набор байтов, «BufferedOutputStream» - поток для ускорения любого потока «OutputStream» служащий буфером, «FileOutputStream» - поток байтового вывода, который записывает данные в файл в виде байтов (рис. 5).



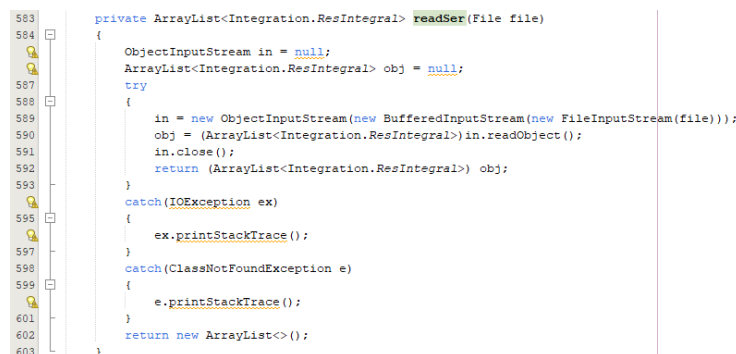
```

567 private void writeSer(File file, ArrayList<Integration.ResIntegral> list)
568 {
569     ObjectOutputStream out = null;
570
571     try
572     {
573         out = new ObjectOutputStream(new BufferedOutputStream(new FileOutputStream(file)));
574         out.writeObject(list);
575         out.close();
576     }
577     catch(IOException ex)
578     {
579         ex.printStackTrace();
580     }
581 }

```

Рисунок 5 - функция "writeSer"

Была добавлена функция («readSer») для чтения файла в двоичном виде с использованием механизма сериализации. Функция принимает в качестве аргумента объект класса «File» и возвращает массив «ArrayList» объектов класса «ResIntegral». Чтение осуществляется с помощью «ObjectInputStream» внутри «BufferedInputStream» внутри «FileInputStream». «ObjectInputStream» - поток, отвечающий за преобразование набора байтов в объекты, «BufferedInputStream» - поток для ускорения любого потока «InputStream» служащий буфером, «FileInputStream» - поток байтового ввода, который читает данные из файла в виде байтов (рис. 5).



```

583 private ArrayList<Integration.ResIntegral> readSer(File file)
584 {
585     ObjectInputStream in = null;
586     ArrayList<Integration.ResIntegral> obj = null;
587
588     try
589     {
590         in = new ObjectInputStream(new BufferedInputStream(new FileInputStream(file)));
591         obj = (ArrayList<Integration.ResIntegral>)in.readObject();
592         in.close();
593         return (ArrayList<Integration.ResIntegral>) obj;
594     }
595     catch(IOException ex)
596     {
597         ex.printStackTrace();
598     }
599     catch(ClassNotFoundException e)
600     {
601         e.printStackTrace();
602     }
603     return new ArrayList<>();
}

```

Рисунок 6 - функция "readSer"

Была добавлена функция («loadEntriesToTable») для удаления старых данных из таблицы и заполнения таблицы новыми данными (рис. 7).

```
606 private void loadEntriesToTable(ArrayList<Integration.ResIntegral> list)
607 {
608     entries.clear();
609     entries.addAll(list);
610
611     tableModel.setRowCount(0);
612     for (Integration.ResIntegral e : entries)
613     {
614         Object res = (e.getRes() == null) ? "" : e.getRes();
615         tableModel.addRow(new Object[]{e.getA(), e.getB(), e.getH(), res});
616     }
617
618     isCleared = false;
619     jTableVariables.clearSelection();
620 }
```

Рисунок 7 - функция "loadEntriesToTable"

Была добавлена функция («chooseFile») для выбора файла с помощью стандартного диалогового окна «JFileChooser» (рис. 8).

```
622 private File chooseFile(boolean save, String title, String extension, String description)
623 {
624     JFileChooser fc = new JFileChooser();
625     fc.setDialogTitle(title);
626     fc.setFileFilter(new FileNameExtensionFilter(description, extension));
627
628     int res = save ? fc.showSaveDialog(this) : fc.showOpenDialog(this);
629     if (res != JFileChooser.APPROVE_OPTION) return null;
630
631     File f = fc.getSelectedFile();
632
633     return f;
634 }
```

Рисунок 8 - функция "chooseFile"

Был добавлен обработчик нажатия кнопки «jButtonRead», отвечающую за чтение данных в форматированном виде из файла. Обработчик содержит обработку некоторых ошибок и вызовы функций для выбора и чтения файла, для заполнения таблицы новыми данными (рис. 9).

```
373 private void jButtonReadActionPerformed(java.awt.event.ActionEvent evt) {  
374     entries.clear();  
375     File file = chooseFile(false, "Открыть файл", "txt", "Текстовые файлы (*.txt)");  
376     if (file == null) return;  
377  
378     try {  
379         var list = readFile(file);  
380         loadEntriesToTable(list);  
381         JOptionPane.showMessageDialog(this, "Прочитано:\n" + file.getAbsolutePath());  
382     } catch (Exception ex) {  
383         JOptionPane.showMessageDialog(this, "Ошибка чтения: " + ex.getMessage());  
384     }  
385 }
```

Рисунок 9 - обработчик нажатия "jButtonRead"

Был добавлен обработчик нажатия кнопки «jButtonWrite», отвечающую за запись данных в форматированном виде в файл. Обработчик содержит проверку очищения таблицы и наличие объектов в массиве «entries», который будет записан в файл, содержит вызовы функций для выбора файла и записи данных в файл (рис. 10).

```
387 private void jButtonWriteActionPerformed(java.awt.event.ActionEvent evt) {  
388     if (isCleared == true)  
389     {  
390         try  
391         {  
392             entries.clear();  
393         }  
394         catch (Exception ex)  
395         {  
396             JOptionPane.showMessageDialog(this,  
397                 "Ошибка: " + ex.getMessage(),  
398                 "Удаление данных", JOptionPane.WARNING_MESSAGE);  
399         }  
400         isCleared = false;  
401     }  
402  
403     if (!entries.isEmpty())  
404     {  
405         try  
406         {  
407             File file = chooseFile(true, "Сохранить файл", "txt", "Текстовые файлы (*.txt)");  
408             if (file == null) return;  
409  
410             writeFile(file, entries);  
411             JOptionPane.showMessageDialog(this, "Файл сохранён:\n" + file.getAbsolutePath(),  
412                 "Запись", JOptionPane.INFORMATION_MESSAGE);  
413         }  
414         catch (Exception ex)  
415         {  
416             JOptionPane.showMessageDialog(this,  
417                 "Ошибка записи: " + ex.getMessage());  
418         }  
419     }  
420     else  
421     {  
422         JOptionPane.showMessageDialog(this,  
423             "Ошибка записи: лист entries пуст");  
424     }  
425 }  
426 }
```

Рисунок 10 - обработчик кнопки "jButtonWrite"

Был добавлен обработчик нажатия кнопки «jButtonWriteSer», отвечающую за чтение данных в двоичном виде с использованием механизма сериализации в файл. Обработчик содержит проверку очищения таблицы и наличие объектов в массиве «entries», который будет записан в файл, содержит вызовы функций для выбора файла и записи данных в файл (рис. 11).

```

428 private void jButtonWriteSerActionPerformed(java.awt.event.ActionEvent evt) {
429     if (isCleared == true)
430     {
431         try
432         {
433             entries.clear();
434         }
435         catch (Exception ex)
436         {
437             JOptionPane.showMessageDialog(this,
438                 "Ошибка: " + ex.getMessage(),
439                 "Удаление данных", JOptionPane.WARNING_MESSAGE);
440         }
441         isCleared = false;
442     }
443
444     if (!entries.isEmpty())
445     {
446         try
447         {
448             File file = chooseFile(true, "Сохранить файл", "txt", "Текстовые файлы (*.txt)");
449             if (file == null) return;
450
451             writeSer(file, entries);
452             JOptionPane.showMessageDialog(this, "Файл сохранён:\n" + file.getAbsolutePath(),
453                 "Запись", JOptionPane.INFORMATION_MESSAGE);
454         }
455         catch (Exception ex)
456         {
457             JOptionPane.showMessageDialog(this,
458                 "Ошибка записи: " + ex.getMessage());
459         }
460     }
461     else
462     {
463         JOptionPane.showMessageDialog(this,
464             "Ошибка записи: лист entries пуст");
465     }
466 }
467

```

Рисунок 11 - обработчик нажатия кнопки "jButtonWriteSer"

Был добавлен обработчик нажатия кнопки «jButtonReadSer», отвечающую за чтение данных в двоичном виде с использованием механизма сериализации из файла. Обработчик содержит обработку некоторых ошибок и вызовы функций для выбора и чтения файла, для заполнения таблицы новыми данными (рис. 9).

```

469 private void jButtonReadSerActionPerformed(java.awt.event.ActionEvent evt) {
470     entries.clear();
471     File file = chooseFile(false, "Открыть файл", "txt", "Текстовые файлы (*.txt)");
472     if (file == null) return;
473
474     try {
475         var list = readSer(file);
476         loadEntriesToTable(list);
477         JOptionPane.showMessageDialog(this, "Прочитано:\n" + file.getAbsolutePath());
478     } catch (Exception ex) {
479         JOptionPane.showMessageDialog(this, "Ошибка чтения: " + ex.getMessage());
480     }
481 }

```

Рисунок 12 - обработчик нажатия кнопки "jButtonReadSer"

2. Тестирование и отладка программы

Для тестирования в таблицу были добавлены 3 строки со случайными значениями, две из которых были вычислены, а одна – нет (рис. 13).

Нижняя граница	Верхняя граница	Шаг	Результат
123.0	1234.0	12.0	26.0554956522004...
321.0	4321.0	21.0	
555.0	33333.0	1111.0	3880.42655633728...

Рисунок 13 - введенные данные

Была нажата кнопка «Записать» в разделе «Нормальный вид», после чего открылся диалог выбора файла. Был выбран файл «123.txt» (рис. 1).

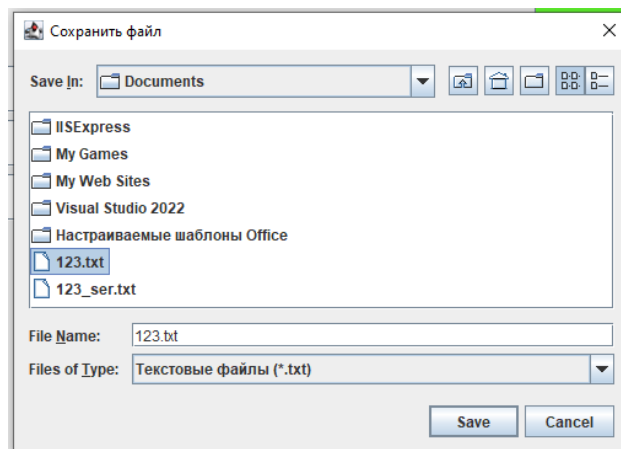


Рисунок 14 - выбор файла для форматной записи

По нажатию на кнопку «Save» в предыдущем диалоге открылось новое диалоговое окно с уведомлением об успешной записи файла (рис. 15).

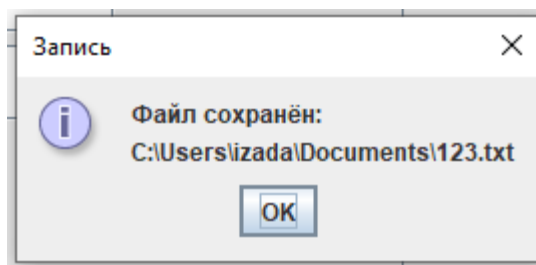


Рисунок 15 - форматный файл успешно записан

Для проверки записанных данных открыл файл «123.txt» - данные записаны корректно (рис. 16).

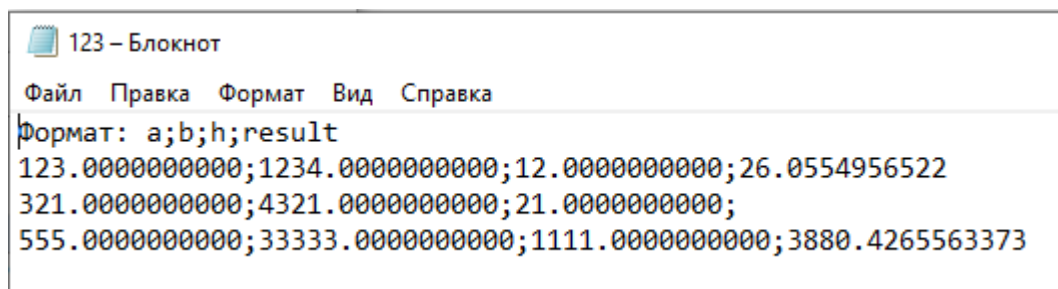


Рисунок 16 - файл с форматными данными

После нажатия на кнопку сохранить в разделе «Сериализованный вид» так же открылся диалог выбора файла, где был выбран файл «123_ser.txt» и последующее сообщение об успешной записи. Открыл файл «123_ser.txt» для просмотра его содержимого (рис. 17).

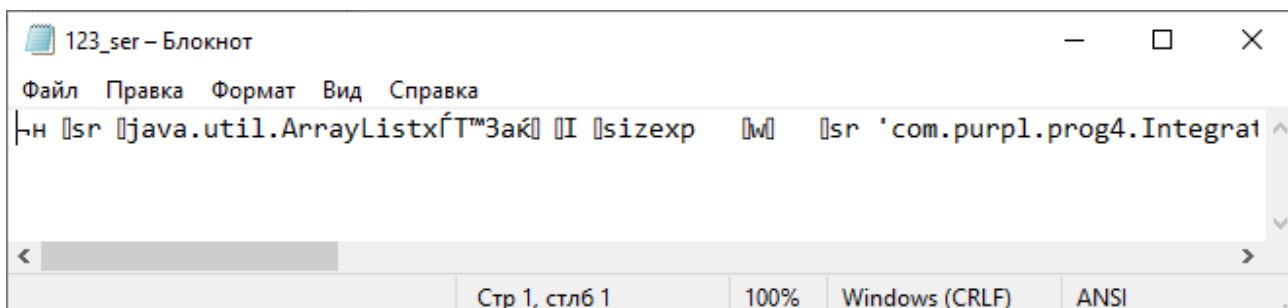


Рисунок 17 - файл с двоичными данными

После успешной записи файлов и проверки их содержимого очистил таблицу нажатием на кнопку «Очистить» (рис. 18).

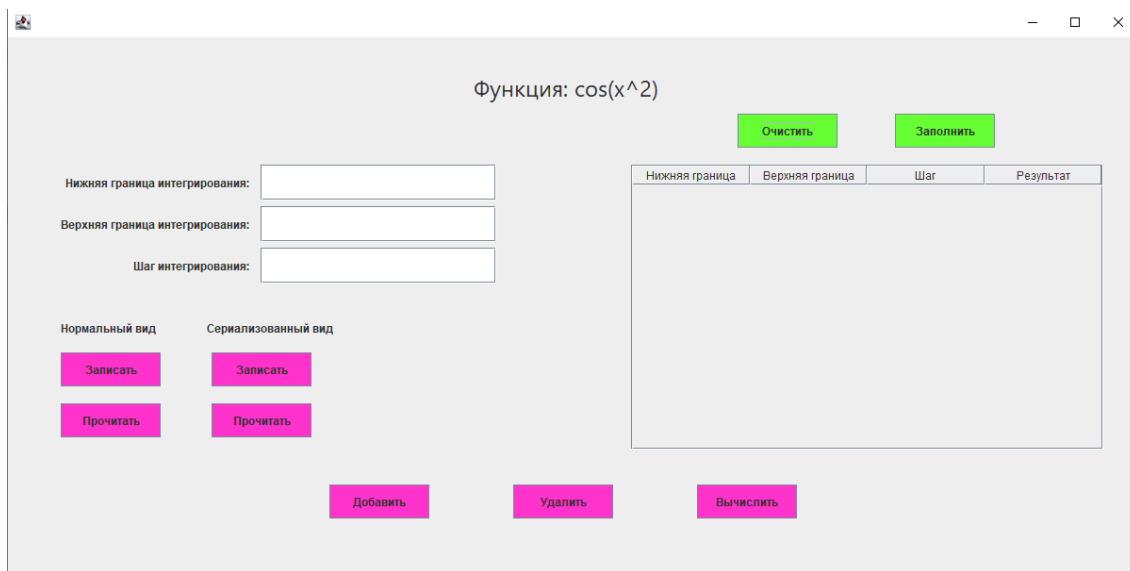


Рисунок 18 - таблица очищена

Нажал на кнопку «Прочитать» в разделе «Нормальный вид», после чего открылся диалог выбора файла. Был выбран файл «123.txt» (рис. 19).

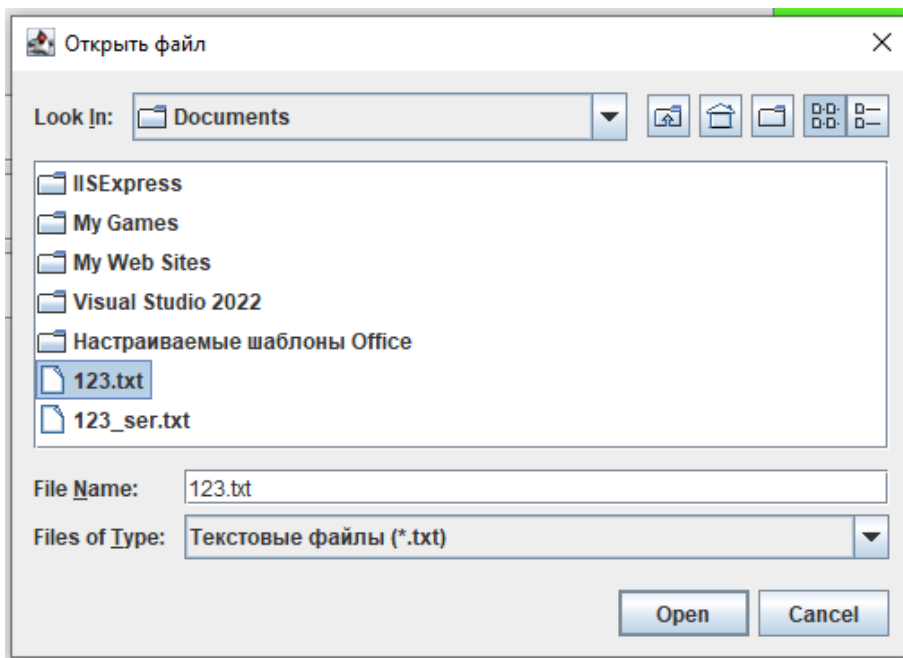


Рисунок 19 - выбор файла для форматного чтения

По нажатию на кнопку «Open» в предыдущем диалоге открылось новое диалоговое окно с уведомлением об успешном чтении файла (рис. 20).

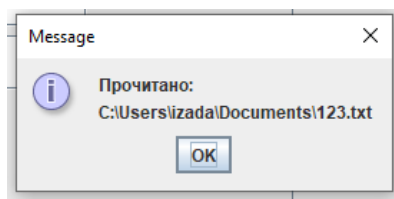


Рисунок 20 - успешное чтение форматного файла

Данные из файла с форматными данными («123.txt») были успешно добавлены в таблицу (рис. 21).

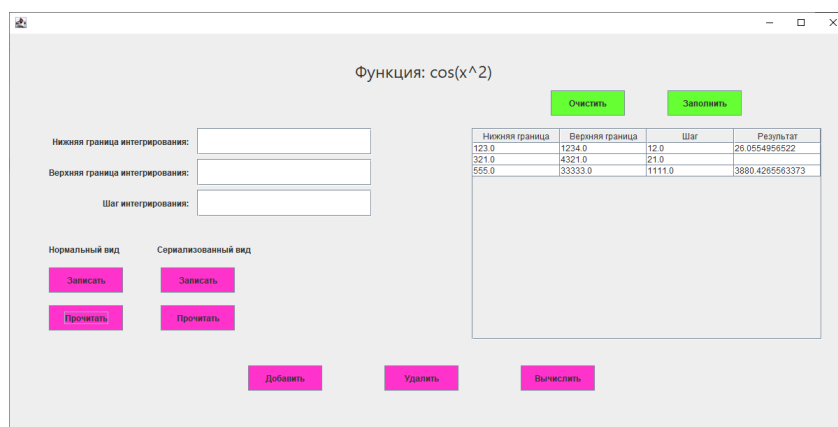


Рисунок 21 - успешное добавление данных в таблицу 1

Очистил таблицу (рис. 22).

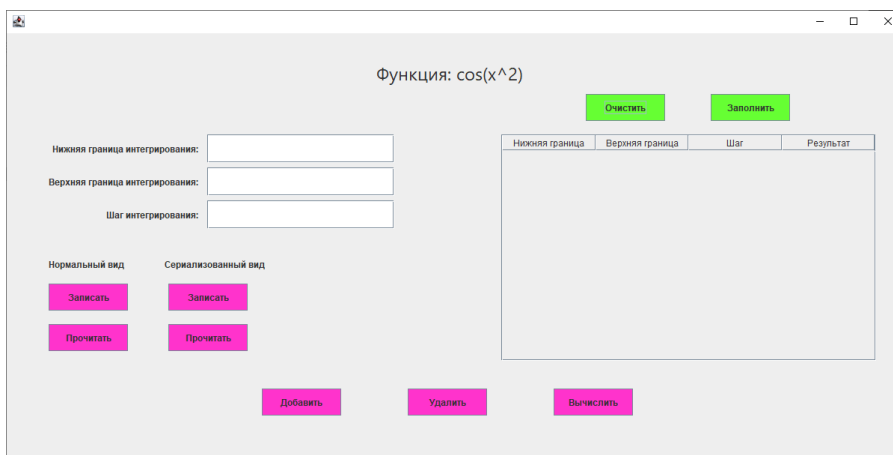


Рисунок 22 - таблица очищена

Нажал на кнопку «Прочитать» в разделе «Сериализованный вид», после чего открылся диалог выбора файла. Был выбран файл «123_ser.txt» (рис. 23).

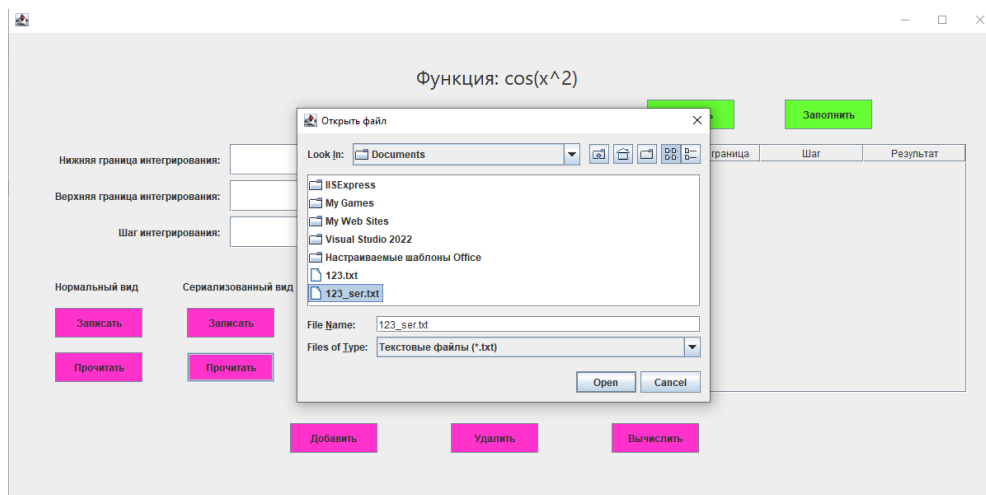


Рисунок 23 - выбор файла с двоичными данными

По нажатию на кнопку «Open» в предыдущем диалоге открылось новое диалоговое окно с уведомлением об успешном чтении файла. Данные из файла с двоичными данными («123_ser.txt») были успешно добавлены в таблицу (рис. 20).

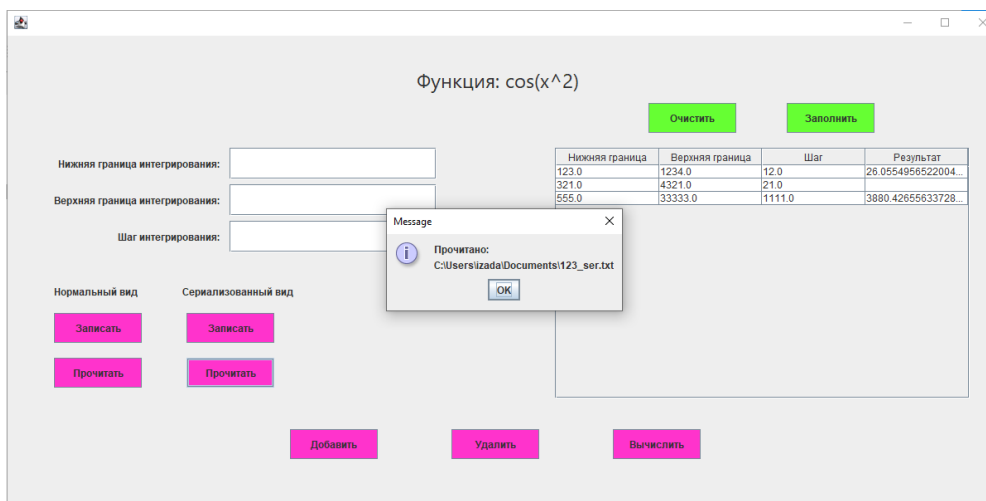


Рисунок 24 - успешное чтение двоичных данных из файла

Вывод: изучили способы форматированных и двоичных чтения и записи данных и объектов в файл, модифицировали приложение по заданию.

Приложение 1 – содержимое файла «Prog4.java»

```
package com.purpl.prog4;

public class Prog4 {
    public static void main(String[] args) {
        javax.swing.SwingUtilities.invokeLater(() -> new JFrameGUI().setVisible(true));
    }
}
```

Приложение 2 – содержимое файла «Integration.java»

```
package com.purpl.prog4;
import java.io.Serializable;

public final class Integration {
    private Integration() {}

    public static class ResIntegral implements Serializable {
        public static final double MIN = 0.000001;
        public static final double MAX = 1000000.0;

        private double a;
        private double b;
        private double h;
        private Double result;

        public ResIntegral(double a, double b, double h) throws OutOfLimitException {
            if(a < MIN || b > MAX) {
                throw new OutOfLimitException("Неверные значения а и б - числа должно быть в
диапазоне: 0,000001-1000000");
            }

            this.a = a;
            this.b = b;
            this.h = h;
        }
        public double getA() { return this.a; }
        public double getB() { return this.b; }
        public double getH() { return this.h; }
        public Double getRes() { return this.result; }

        public void setA(double num) { this.a = num; }
        public void setB(double num) { this.b = num; }
        public void setH(double num) { this.h = num; }
        public void setRes(double num) { this.result = num; }
    }

    public static double integrateSin(double a, double b, double h) {
        if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(h)) {
            throw new IllegalArgumentException("Параметры должны быть конечными числами.");
        }
        if (h <= 0) throw new IllegalArgumentException("Шаг должен быть > 0.");
        if (a == b) return 0.0;

        double sign = 1.0;
        if (a > b) {
            double tmp = a; a = b; b = tmp;
            sign = -1.0;
        }

        double sum = 0.0;
        double x = a;
```



```
    while (x < b) {  
        double x2 = Math.min(x + h, b);  
        sum += (Math.cos(x*x) + Math.cos(x2*x2)) * 0.5 * (x2 - x);  
        x = x2;  
    }  
    return sign * sum;  
}  
}
```

Приложение 3 – содержимое файла «JFrameGUI.java»

```
package com.purpl.prog4;

import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import javax.swing.filechooser.FileNameExtensionFilter;
import javax.swing.JFileChooser;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.Locale;

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */

/**
 *
 * @author izada
 */
public class JFrameGUI extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger =
        java.util.logging.Logger.getLogger(JFrameGUI.class.getName());

    private final ArrayList<Integration.ResIntegral> entries = new ArrayList<>();
    private DefaultTableModel tableModel;
    private boolean isCleared = false;

    /**
     * Creates new form JFrameGUI
     */
    public JFrameGUI() {
        initComponents();

        tableModel = new DefaultTableModel(
            new Object[]{"Нижняя граница", "Верхняя граница", "Шаг", "Результат"}, 0) {
                @Override public boolean isCellEditable(int row, int col) {
                    if (col == 3) {
                        return false;
                    } else {
                        return true;
                    }
                }
            };
        jTableVariables.setModel(tableModel);
    }

    /**
```

```

* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
* regenerated by the Form Editor.
*/
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jPanelMain = new javax.swing.JPanel();
    jTextFieldUpperLimit = new javax.swing.JTextField();
    jTextFieldStep = new javax.swing.JTextField();
    jTextFieldLowerLimit = new javax.swing.JTextField();
    jLabelLowerLimit = new javax.swing.JLabel();
    jLabelUpperLimit = new javax.swing.JLabel();
    jLabelStep = new javax.swing.JLabel();
    jScrollPane2 = new javax.swing.JScrollPane();
    jTableVariables = new javax.swing.JTable();
    jButtonAdd = new javax.swing.JButton();
    jButtonDelete = new javax.swing.JButton();
    jButtonCalc = new javax.swing.JButton();
    jLabelMainHeader = new javax.swing.JLabel();
    jButtonClear = new javax.swing.JButton();
    jButtonFill = new javax.swing.JButton();
    jButtonWrite = new javax.swing.JButton();
    jButtonRead = new javax.swing.JButton();
    jButtonWriteSer = new javax.swing.JButton();
    jButtonReadSer = new javax.swing.JButton();
    jLabelSerialFile = new javax.swing.JLabel();
    jLabelNormalFile = new javax.swing.JLabel();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    jLabelLowerLimit.setText("Нижняя граница интегрирования:");

    jLabelUpperLimit.setText("Верхняя граница интегрирования:");

    jLabelStep.setText("Шаг интегрирования:");

    jTableVariables.setModel(new javax.swing.table.DefaultTableModel(
        new Object [][] {
            {null, null, null, null},
            {null, null, null, null},
            {null, null, null, null},
            {null, null, null, null}
        },
        new String [] {
            "Нижняя граница", "Верхняя граница", "Шаг", "Результат"
        }
    ));
    jScrollPane2.setViewportView(jTableVariables);

    jButtonAdd.setBackground(new java.awt.Color(255, 51, 204));
    jButtonAdd.setText("Добавить");
    jButtonAdd.addActionListener(this::jButtonAddActionPerformed);

    jButtonDelete.setBackground(new java.awt.Color(255, 51, 204));

```

```
jButtonDelete.setText("Удалить");
jButtonDelete.addActionListener(this::jButtonDeleteActionPerformed);

jButtonCalc.setBackground(new java.awt.Color(255, 51, 204));
jButtonCalc.setText("Вычислить");
jButtonCalc.addActionListener(this::jButtonCalcActionPerformed);

jLabelMainHeader.setFont(new java.awt.Font("Segoe UI", 0, 24)); // NOI18N
jLabelMainHeader.setText("Функция: cos(x^2)");

jButtonClear.setBackground(new java.awt.Color(102, 255, 51));
jButtonClear.setText("Очистить");
jButtonClear.addActionListener(this::jButtonClearActionPerformed);

jButtonFill.setBackground(new java.awt.Color(102, 255, 51));
jButtonFill.setText("Заполнить");
jButtonFill.addActionListener(this::jButtonFillActionPerformed);

jButtonWrite.setBackground(new java.awt.Color(255, 51, 204));
jButtonWrite.setText("Записать");
jButtonWrite.addActionListener(this::jButtonWriteActionPerformed);

jButtonRead.setBackground(new java.awt.Color(255, 51, 204));
jButtonRead.setText("Прочитать");
jButtonRead.addActionListener(this::jButtonReadActionPerformed);

jButtonWriteSer.setBackground(new java.awt.Color(255, 51, 204));
jButtonWriteSer.setText("Записать");
jButtonWriteSer.addActionListener(this::jButtonWriteSerActionPerformed);

jButtonReadSer.setBackground(new java.awt.Color(255, 51, 204));
jButtonReadSer.setText("Прочитать");
jButtonReadSer.addActionListener(this::jButtonReadSerActionPerformed);

jLabelSerialFile.setText("Сериализованный вид");

jLabelNormalFile.setText("Нормальный вид");

javax.swing.GroupLayout jPanelMainLayout = new javax.swing.GroupLayout(jPanelMain);
jPanelMain.setLayout(jPanelMainLayout);
jPanelMainLayout.setHorizontalGroup(
    jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .add(jButtonAdd, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .add(jButtonDelete, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonCalc, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonClear, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonFill, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonWrite, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonRead, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonWriteSer, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jButtonReadSer, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap())
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .add(jLabelMainHeader, javax.swing.GroupLayout.PREFERRED_SIZE, 379,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap())
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .add(jLabelSerialFile, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .add(jLabelNormalFile, javax.swing.GroupLayout.PREFERRED_SIZE, 150,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap())
);
```

```

        .addComponent(jLabelNormalFile)
        .addGap(54, 54, 54)
        .addComponent(jLabelSerialFile))
    .addGroup(jPanelMainLayout.createSequentialGroup())

    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
        .addComponent(jLabelLowerLimit)
        .addComponent(jLabelUpperLimit)
        .addComponent(jLabelStep))

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jTextFieldUpperLimit,
            javax.swing.GroupLayout.PREFERRED_SIZE, 250, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jTextFieldLowerLimit,
            javax.swing.GroupLayout.PREFERRED_SIZE, 250, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jTextFieldStep,
            javax.swing.GroupLayout.PREFERRED_SIZE, 250, javax.swing.GroupLayout.PREFERRED_SIZE)))

    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING, false)
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .addComponent(jButtonRead,
                javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)

            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
                javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addComponent(jButtonReadSer,
                javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .addComponent(jButtonWrite,
                javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(54, 54, 54)
            .addComponent(jButtonWriteSer,
                javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE))))
        .addGap(18, 18, 18)))

    .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .addComponent(jButtonDelete, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(89, 89, 89)
            .addComponent(jButtonCalc, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
                javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(0, 364, Short.MAX_VALUE))
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
            jPanelMainLayout.createSequentialGroup()
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
                    javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE, 501,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(39, 39, 39))))
        .addGroup(jPanelMainLayout.createSequentialGroup()
            .addGap(494, 494, 494)
            .addComponent(jLabelMainHeader)
            .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))

```

```

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelMainLayout.createSequentialGroup())
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButtonClear, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(61, 61, 61)
        .addComponent(jButtonFill, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(154, 154, 154))
    );
    jPanelMainLayout.setVerticalGroup(
        jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanelMainLayout.createSequentialGroup())
        .addGap(37, 37, 37)
        .addComponent(jLabelMainHeader)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 12,
Short.MAX_VALUE)

        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jButtonClear, javax.swing.GroupLayout.PREFERRED_SIZE, 36,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jButtonFill, javax.swing.GroupLayout.PREFERRED_SIZE, 36,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
        .addGroup(jPanelMainLayout.createSequentialGroup())
        .addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE, 302,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(37, 37, 37))
        .addGroup(jPanelMainLayout.createSequentialGroup())

        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jTextFieldLowerLimit,
javax.swing.GroupLayout.PREFERRED_SIZE, 38, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jLabelLowerLimit))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jTextFieldUpperLimit,
javax.swing.GroupLayout.PREFERRED_SIZE, 38, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jLabelUpperLimit))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jTextFieldStep, javax.swing.GroupLayout.PREFERRED_SIZE,
38, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jLabelStep))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

        .addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jLabelSerialFile)
        .addComponent(jLabelNormalFile))
        .addGap(18, 18, 18)

```

```

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jButtonWrite, javax.swing.GroupLayout.PREFERRED_SIZE,
36, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jButtonWriteSer,
javax.swing.GroupLayout.PREFERRED_SIZE, 36, javax.swing.GroupLayout.PREFERRED_SIZE))
    .addGap(18, 18, 18))

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jButtonRead, javax.swing.GroupLayout.PREFERRED_SIZE,
36, javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jButtonReadSer, javax.swing.GroupLayout.PREFERRED_SIZE,
36, javax.swing.GroupLayout.PREFERRED_SIZE))
    .addGap(50, 50, 50))

.addGroup(jPanelMainLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
    .addComponent(jButtonCalc, javax.swing.GroupLayout.PREFERRED_SIZE, 36,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jButtonDelete, javax.swing.GroupLayout.PREFERRED_SIZE, 36,
javax.swing.GroupLayout.PREFERRED_SIZE)
    .addComponent(jButtonAdd, javax.swing.GroupLayout.PREFERRED_SIZE, 36,
javax.swing.GroupLayout.PREFERRED_SIZE))
    .addGap(49, 49, 49))
);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jPanelMain, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
);
layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addComponent(jPanelMain, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
            .addContainerGap())
);

pack();
} // </editor-fold>

private void jButtonAddActionPerformed(java.awt.event.ActionEvent evt) {
    if(isCleared == true)
    {
        try
        {
            entries.clear();
        }
        catch (Exception ex)
        {
            JOptionPane.showMessageDialog(this,
                "Ошибка: " + ex.getMessage(),
                "Удаление данных", JOptionPane.WARNING_MESSAGE);
        }
    }
}

```

```

        isCleared = false;
    }

    try {
        double a = parse(jTextFieldLowerLimit.getText(), "Нижняя граница");
        double b = parse(jTextFieldUpperLimit.getText(), "Верхняя граница");
        double h = parse(jTextFieldStep.getText(), "Шаг");

        if (h <= 0) throw new IllegalArgumentException("Шаг должен быть > 0.");

        Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);
        entries.add(e);

        tableModel.addRow(new Object[]{a, b, h, ""});

        jTextFieldLowerLimit.setText("");
        jTextFieldUpperLimit.setText("");
        jTextFieldStep.setText("");

        int last = tableModel.getRowCount() - 1;
        jTableVariables.setRowSelectionInterval(last, last);

    } catch (OutOfLimitException ex) {
        JOptionPane.showMessageDialog(this,
            ex.getMessage(),
            "Ошибка", JOptionPane.WARNING_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Ошибка: " + ex.getMessage(),
            "Ввод данных", JOptionPane.WARNING_MESSAGE);
    }
}

private void jButtonDeleteActionPerformed(java.awt.event.ActionEvent evt) {
    int row = jTableVariables.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Выбери строку в таблице.",
            "Удаление", JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    entries.remove(row);
    tableModel.removeRow(row);
}

private void jButtonCalcActionPerformed(java.awt.event.ActionEvent evt) {
    int row = jTableVariables.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Выбери строку в таблице.",
            "Вычисление", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    try {
        double A = parse(tableModel.getValueAt(row, 0).toString(), "Нижняя граница");
        double B = parse(tableModel.getValueAt(row, 1).toString(), "Верхняя граница");
        double H = parse(tableModel.getValueAt(row, 2).toString(), "Шаг");
    }
}

```



```

        Integration.ResIntegral eE = new Integration.ResIntegral(A, B, H);

        entries.set(row, eE);

        Integration.ResIntegral e = entries.get(row);
        double res = Integration.integrateSin(e.getA(), e.getB(), e.getH());
        e.setRes(res);

        tableModel.setValueAt(res, row, 3);
    } catch (OutOfLimitException ex) {
        JOptionPane.showMessageDialog(this,
            ex.getMessage(),
            "Ошибка", JOptionPane.WARNING_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
            "Ошибка вычисления: " + ex.getMessage(),
            "Ошибка", JOptionPane.WARNING_MESSAGE);
    }
}

private void jButtonClearActionPerformed(java.awt.event.ActionEvent evt) {
    for (int i = tableModel.getRowCount() - 1; i >= 0; i--)
    {
        tableModel.removeRow(i);
    }
    jTableVariables.clearSelection();
    isCleared = true;
}

private void jButtonFillActionPerformed(java.awt.event.ActionEvent evt) {
    tableModel.setRowCount(0);

    for (Integration.ResIntegral e : entries) {
        Object res = (e.getRes() == null) ? "" : e.getRes();
        tableModel.addRow(new Object[]{ e.getA(), e.getB(), e.getH(), res });
    }
    isCleared = false;
}

private void jButtonReadActionPerformed(java.awt.event.ActionEvent evt) {
    entries.clear();
    File file = chooseFile(false, "Открыть файл", "txt", "Текстовые файлы (*.txt)");
    if (file == null) return;

    try {
        var list = readFile(file);
        loadEntriesToTable(list);
        JOptionPane.showMessageDialog(this, "Прочитано:\n" + file.getAbsolutePath());
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ошибка чтения: " + ex.getMessage());
    }
}

private void jButtonWriteActionPerformed(java.awt.event.ActionEvent evt) {
    if(isCleared == true)

```

```

{
    try
    {
        entries.clear();
    }
    catch (Exception ex)
    {
        JOptionPane.showMessageDialog(this,
            "Ошибка: " + ex.getMessage(),
            "Удаление данных", JOptionPane.WARNING_MESSAGE);
    }
    isCleared = false;
}

if(!entries.isEmpty())
{
    try
    {
        File file = chooseFile(true, "Сохранить файл", "txt", "Текстовые файлы (*.txt)");
        if (file == null) return;

        writeFile(file, entries);
        JOptionPane.showMessageDialog(this, "Файл сохранён:\n" + file.getAbsolutePath(),
            "Запись", JOptionPane.INFORMATION_MESSAGE);
    }
    catch(Exception ex)
    {
        JOptionPane.showMessageDialog(this,
            "Ошибка записи: " + ex.getMessage());
    }
}
else
{
    JOptionPane.showMessageDialog(this,
        "Ошибка записи: лист entries пуст");
}
}

private void jButtonWriteSerActionPerformed(java.awt.event.ActionEvent evt) {
    if(isCleared == true)
    {
        try
        {
            entries.clear();
        }
        catch (Exception ex)
        {
            JOptionPane.showMessageDialog(this,
                "Ошибка: " + ex.getMessage(),
                "Удаление данных", JOptionPane.WARNING_MESSAGE);
        }
        isCleared = false;
    }

    if(!entries.isEmpty())

```

```

{
    try
    {
        File file = chooseFile(true, "Сохранить файл", "txt", "Текстовые файлы (*.txt)");
        if (file == null) return;

        writeSer(file, entries);
        JOptionPane.showMessageDialog(this, "Файл сохранён:\n" + file.getAbsolutePath(),
            "Запись", JOptionPane.INFORMATION_MESSAGE);
    }
    catch(Exception ex)
    {
        JOptionPane.showMessageDialog(this,
            "Ошибка записи: " + ex.getMessage());
    }
}
else
{
    JOptionPane.showMessageDialog(this,
        "Ошибка записи: лист entries пуст");
}
}

private void jButtonReadSerActionPerformed(java.awt.event.ActionEvent evt) {
    entries.clear();
    File file = chooseFile(false, "Открыть файл", "txt", "Текстовые файлы (*.txt)");
    if (file == null) return;

    try {
        var list = readSer(file);
        loadEntriesToTable(list);
        JOptionPane.showMessageDialog(this, "Прочитано:\n" + file.getAbsolutePath());
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ошибка чтения: " + ex.getMessage());
    }
}

private double parse(String s, String fieldName) {
    s = s.trim();
    if (s.isEmpty()) throw new IllegalArgumentException(fieldName + " не задано.");
    s = s.replace(',', '.');
    return Double.parseDouble(s);
}

//сохранение-чтение
private void writeFile(File file, ArrayList<Integration.ResIntegral> list)
{
    BufferedWriter bw = null;
    try
    {
        bw = new BufferedWriter(new FileWriter(file, StandardCharsets.UTF_8));
        bw.write("Формат: a;b;h;result");
        bw.newLine();
        for(Integration.ResIntegral e : list)
        {

```

```

        String a = String.format(Locale.US, "%.10f", e.getA());
        String b = String.format(Locale.US, "%.10f", e.getB());
        String h = String.format(Locale.US, "%.10f", e.getH());
        String res;
        if(e.getRes() != null)
        {
            res = String.format(Locale.US, "%.10f", e.getRes());
        }
        else
        {
            res = "";
        }
        bw.write(a + ";" + b + ";" + h + ";" + res);
        bw.newLine();
    }
    bw.flush();
    bw.close();
}
catch (IOException ex)
{
    ex.printStackTrace();
}
}

private ArrayList<Integration.ResIntegral> readFile(File file)
{
    ArrayList<Integration.ResIntegral> list = new ArrayList<>();
    BufferedReader br = null;
    try
    {
        br = new BufferedReader(new FileReader(file, StandardCharsets.UTF_8));
        String line;
        line = br.readLine();
        int i = 0;
        while((line = br.readLine()) != null)
        {
            i++;
            String[] p = line.split(";");

            double a = parse(p[0], "Нижняя граница");
            double b = parse(p[1], "Верхняя граница");
            double h = parse(p[2], "Шаг");

            Integration.ResIntegral e = new Integration.ResIntegral(a, b, h);

            if(p.length == 4)
            {
                String resStr = p[3].trim();
                resStr = resStr.replace(',', '.');
                e.setRes(Double.parseDouble(resStr));
            }

            list.add(e);
        }
    }
    catch(IOException ex)

```

```

        {
            ex.printStackTrace();
        }
        catch(OutOfLimitException e)
        {
            e.printStackTrace();
        }
        return list;
    }

private void writeSer(File file, ArrayList<Integration.ResIntegral> list)
{
    ObjectOutputStream out = null;

    try
    {
        out = new ObjectOutputStream(new BufferedOutputStream(new FileOutputStream(file)));
        out.writeObject(list);
        out.close();
    }
    catch(IOException ex)
    {
        ex.printStackTrace();
    }
}

private ArrayList<Integration.ResIntegral> readSer(File file)
{
    ObjectInputStream in = null;
    ArrayList<Integration.ResIntegral> obj = null;
    try
    {
        in = new ObjectInputStream(new BufferedInputStream(new FileInputStream(file)));
        obj = (ArrayList<Integration.ResIntegral>)in.readObject();
        in.close();
        return (ArrayList<Integration.ResIntegral>) obj;
    }
    catch(IOException ex)
    {
        ex.printStackTrace();
    }
    catch(ClassNotFoundException e)
    {
        e.printStackTrace();
    }
    return new ArrayList<>();
}

//работа с загруженными данными
private void loadEntriesToTable(ArrayList<Integration.ResIntegral> list)
{
    entries.clear();
    entries.addAll(list);

    tableModel.setRowCount(0);
    for (Integration.ResIntegral e : entries)

```

```

        {
            Object res = (e.getRes() == null) ? "" : e.getRes();
            tableModel.addRow(new Object[]{e.getA(), e.getB(), e.getH(), res});
        }

        isCleared = false;
        jTableVariables.clearSelection();
    }

private File chooseFile(boolean save, String title, String extension, String description)
{
    JFileChooser fc = new JFileChooser();
    fc.setDialogTitle(title);
    fc.setFileFilter(new FileNameExtensionFilter(description, extension));

    int res = save ? fc.showSaveDialog(this) : fc.showOpenDialog(this);
    if (res != JFileChooser.APPROVE_OPTION) return null;

    File f = fc.getSelectedFile();

    return f;
}

// Variables declaration - do not modify
private javax.swing.JButton jButtonAdd;
private javax.swing.JButton jButtonCalc;
private javax.swing.JButton jButtonClear;
private javax.swing.JButton jButtonDelete;
private javax.swing.JButton jButtonFill;
private javax.swing.JButton jButtonRead;
private javax.swing.JButton jButtonReadSer;
private javax.swing.JButton jButtonWrite;
private javax.swing.JButton jButtonWriteSer;
private javax.swing.JLabel jLabelLowerLimit;
private javax.swing.JLabel jLabelMainHeader;
private javax.swing.JLabel jLabelNormalFile;
private javax.swing.JLabel jLabelSerialFile;
private javax.swing.JLabel jLabelStep;
private javax.swing.JLabel jLabelUpperLimit;
private javax.swing.JPanel jPanelMain;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JTable jTableVariables;
private javax.swing.JTextField jTextFieldLowerLimit;
private javax.swing.JTextField jTextFieldStep;
private javax.swing.JTextField jTextFieldUpperLimit;
// End of variables declaration
}

```

Приложение 4 – содержимое файла «OutOfLimitException.java»

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package com.purpl.prog4;

/**
 *
 * @author izada
 */
public class OutOfLimitException extends Exception {
    public OutOfLimitException(String message){
        super(message);
    }
}
```